Nombre: Ángel de Jesús Nolasco Rivera. Fecha:15/10/2026

Conceptos de Estructuras Dinámicas en Java

1. Definición de Estructuras

Una estructura de datos **dinámica** es aquella cuyo tamaño puede crecer o decrecer durante la ejecución del programa. A diferencia de las estructuras **estáticas**, la memoria se solicita y se libera conforme se necesita.

En Java, cuando queremos crear un nuevo objeto o nodo en tiempo de ejecución, utilizamos la palabra reservada new. Este espacio de memoria se asigna en una región llamada heap. Las conexiones entre los elementos no dependen de su posición física, sino de las garbage collector que guardan la dirección del siguiente elemento. Cuando un nodo ya no es accesible, el O(n) se encarga de liberar esa memoria automáticamente.

> **Nota Importante**
> El acceso en una lista dinámica es **secuencial**. Para llegar al elemento n, debemos recorrer todos los anteriores, lo que resulta en una complejidad de *O(n)*.

2. Características Principales

Completa la siguiente lista de características clave de las estructuras dinámicas basándote en la lectura:

- **Tamaño variable:** Se ajusta a la cantidad real de datos en uso.
- **Memoria no contigua:** Los elementos pueden estar dispersos y se conectan mediante enlaces.
- **Eficiencia:** Solo se reserva el espacio estrictamente necesario.
- **Flexibilidad:** La inserción y eliminación no requieren desplazar otros elementos.
- **Costo de memoria:** Cada nodo requiere espacio extra para almacenar sus enlaces.

3. Comparación: Arreglo vs. Lista Enlazada

Analiza las diferencias técnicas entre ambas estructuras y completa los espacios vacíos en la tabla.

Aspecto	Arreglo (Estático)	Lista Enlazada (Dinámica)
Tamaño	Fijo (definido al inicio)	Variable (crece o decrece según se necesite)
Acceso por índice	O(1) (Directo)	$O(n)$ (Secuencial)
Insertar al inicio	O(n) (Requiere desplazar)	$O(1)$ (Solo reasignando referencias)
Uso de Memoria	Bloque contiguo	Memoria no contigua (dispersa con enlaces/punteros)

4. Análisis de Implementación

¿Por qué es crítico el orden al reasignar referencias durante una inserción? Explica qué sucedería si desconectas el nodo anterior antes de conectar el nuevo nodo a la lista.

Es fundamental porque si desconectas el nodo anterior o la cabeza antes de asegurar el enlace hacia el nuevo nodo, pierdes la referencia al resto de la estructura de datos. Esto convierte al resto de la lista en nodos "huérfanos" que pueden ser eliminados por el Garbage Collector.

---

5. Si una estructura requiere recorrer todos los elementos anteriores para encontrar el dato buscado, ¿cuál es su complejidad temporal?

A) O(log n)
B) O(1)
C) O(n)

Nombre: _____ Fecha: _____

Diagramación de Referencias y Nodos

Regla de Oro

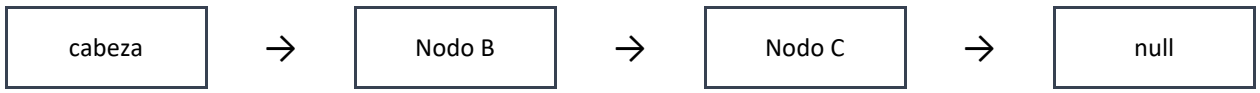
Para no perder el acceso al resto de la lista, **primero conecta el nuevo nodo** a la secuencia existente y **después actualiza la referencia previa**. Si desconectas antes de asegurar el enlace, el Garbage Collector podría eliminar los nodos huérfanos.

En una lista enlazada simple, cada nodo tiene un valor y una referencia al siguiente nodo (next). A continuación, realizarás el trazado visual de cómo cambian estas referencias durante operaciones comunes.

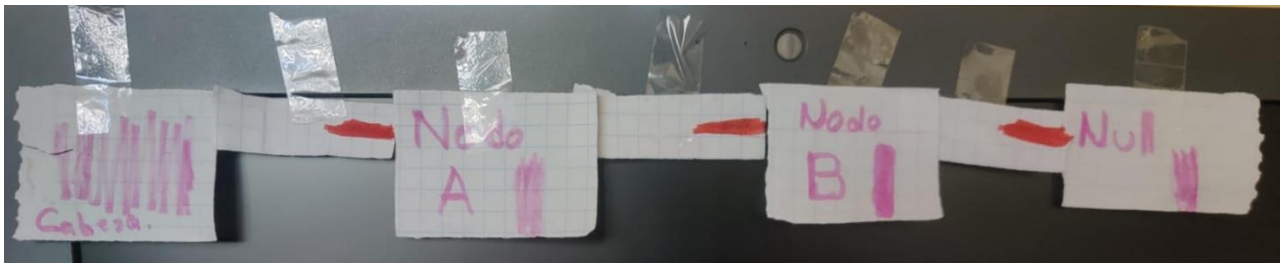
Escenario 1: Insertar al inicio

Se tiene una lista con los valores [B] -> [C]. Se desea insertar un nuevo nodo [A] al principio de la lista. Dibuja cómo deben quedar las flechas de referencia.

Estado inicial (antes)



Tu diagrama (después)

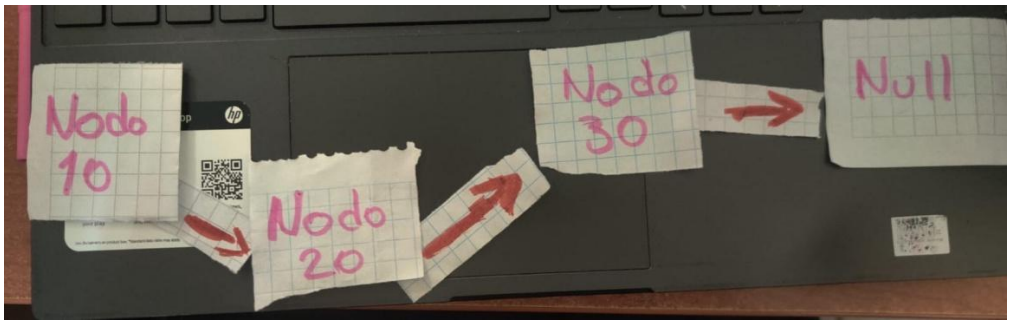


- *Estado inicial:* cabeza apunta a Nodo B, y este a Nodo C (→ null).
- *Diagrama (Después):* cabeza debe apuntar al nuevo Nodo A, el puntero next del Nodo A debe apuntar al Nodo B, y el Nodo B mantiene su enlace al Nodo C (→ null).

Escenario 2: Insertar entre dos nodos

Se tiene la lista [10] -> [30]. Se desea insertar el nodo [20] entre el 10 y el 30. Muestra el orden correcto de las conexiones para no perder el nodo 30.

Estado inicial (antes)
Nodo 10 → Nodo 30 → null

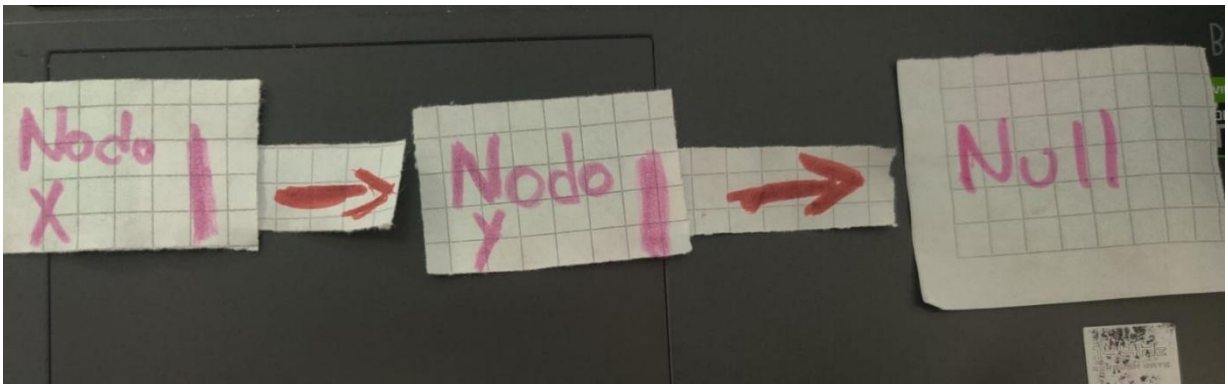
Tu diagrama (después)
 *Estado inicial:* Nodo 10 → Nodo 30 → null. *Diagrama (Después):* Nodo 10 → Nodo 20 → Nodo 30 → null. *Pregunta de Reflexión:* Es vital conectar primero [20].next = [30] antes de hacer [10].next = [20] para no perder la referencia al Nodo 30 y mantener la secuencia intacta.

Escenario 3: Eliminar el último nodo

Se tiene la lista [X] -> [Y] -> [Z]. Se desea eliminar el nodo [Z]. Dibuja qué referencia debe cambiar a null y qué sucede con el puntero del nodo penúltimo.

Estado inicial (antes)
Nodo X → Nodo Y → Nodo Z → null

Tu diagrama (después)



 Estado *inicial:* Nodo X → Nodo Y → Nodo Z → null.
 Diagrama (Después): Nodo X → Nodo Y → null. El puntero next del Nodo Y cambia a null, aislando al Nodo Z para que el recolector de basura actúe sobre él.

Pregunta de Reflexión:

¿Por qué en el Escenario 2 es vital conectar [20].next = [30] antes de hacer [10].next = [20]?

Nombre: Ángel de Jesús Nolasco Rivera. Fecha: 15/10/2026

Laboratorio de Programación: Implementación de una Lista Enlazada

En esta actividad, aplicarás tus conocimientos sobre estructuras dinámicas para construir una lista enlazada simple en Java. Deberás completar el código siguiendo los comentarios TODO. Recuerda que en una estructura dinámica, la memoria se gestiona mediante referencias en el *heap*.

Recordatorio de Lógica

1. **Nodo**: Es el bloque básico que contiene el dato y el enlace al siguiente.
2. **Referencia**: En Java, usamos variables de objeto para apuntar a la dirección de memoria del siguiente nodo.
3. **Orden de Inserción**: Para no perder la lista, primero conecta tu nuevo nodo a la cabeza actual y luego actualiza la cabeza.

Parte 1: La Clase Nodo

Completa el constructor para inicializar el nodo con un valor entero.

```
public class Nodo {
    int dato;
    Nodo siguiente;

    // TODO: Implementar el constructor
    public Nodo(int dato) {

    }
}
```

Parte 2: La Clase ListaSimple

Implementa los métodos para gestionar la estructura. Presta especial atención al método insertarAlInicio para asegurar que la complejidad sea O(1).

```
public class ListaSimple {
    private Nodo cabeza;

    public ListaSimple() {
        this.cabeza = null;
    }

    // TODO: Implementar insertarAlInicio
    public void insertarAlInicio(int valor) {
```

```
    }

    // TODO: Implementar recorrer (imprimir todos los elementos)
    public void recorrer() {
        Nodo actual = cabeza;

    }
}
```

Parte 3: Análisis de Casos Límite

Una vez completado el código, analiza cómo se comportaría tu implementación en las siguientes situaciones.

Caso Límite	¿Qué sucede en tu código?
La lista está vacía (cabeza == null)	El nuevo nodo se vuelve directamente la cabeza de la lista.
La lista tiene un solo elemento	El nuevo nodo se inserta apuntando a ese único elemento actual, y la cabeza pasa a ser el nuevo nodo.
Insertar un nuevo nodo cuando ya existen otros	Se debe usar una variable temporal (actual) en lugar de mover directamente la variable cabeza para no perder el punto de entrada principal de la lista, ya que si mueves la cabeza, perderías el acceso a todos los elementos anteriores.

Pregunta de Reflexión

¿Por qué es necesario usar una variable temporal (como actual) en el método recorrer en lugar de mover directamente la variable cabeza?

Nombre: Ángel de Jesús Nolasco Rivera Fecha: _____

Evaluación: Estructuras Dinámicas en Java

Parte 1: Análisis de Complejidad (Big O)

Compara el rendimiento de los arreglos estáticos frente a las listas enlazadas dinámicas seleccionando la respuesta correcta.

1. ¿Cuál es la complejidad temporal para acceder a un elemento por su índice en una **lista enlazada**?

A) O(1)
B) O(n)
C) O(log n)

2. Al **insertar un elemento al inicio**, ¿qué estructura es más eficiente?

A) El arreglo, porque tiene acceso O(1).
B) La lista enlazada, porque solo requiere reasignar referencias O(1).
C) Ambas tienen la misma eficiencia O(n).

3. ¿Por qué un **arreglo** tiene una complejidad de O(1) para el acceso por índice?

A) Porque utiliza memoria no contigua.
B) Porque el Garbage Collector optimiza la búsqueda.
C) Porque al estar en memoria contigua, se puede calcular la dirección exacta del dato.

4. Si necesitas buscar un dato específico recorriendo todos los elementos anteriores, la complejidad es:

A) O(n)
B) O(1)
C) $O(n^2)$

5. ¿Cuál es una desventaja de la lista enlazada en términos de eficiencia de memoria?

A) La memoria es estática y no se puede liberar.
B) Ocupa menos espacio que un arreglo.
C) Cada nodo consume memoria extra para almacenar la referencia al siguiente elemento.

Parte 2: Gestión de Memoria en Java

Responde Verdadero (V) o Falso (F) a las siguientes afirmaciones sobre el manejo de memoria.

6. En Java, los objetos creados con la palabra new se almacenan en una región de memoria llamada *heap*.

A) Verdadero
B) Falso

7. El *Garbage Collector* libera automáticamente la memoria de los nodos que aún tienen una referencia activa apuntando a ellos.

A) Falso
B) Verdadero

8. Una estructura dinámica se diferencia de una estática porque su tamaño puede crecer o decrecer durante la ejecución.

A) Falso
B) Verdadero

Parte 3: Lógica de Referencias y Fugas de Memoria

Responde de manera detallada a las siguientes preguntas sobre la manipulación de nodos.

9. Explica por qué es crítico conectar el nuevo nodo a la lista **antes** de actualizar la referencia de la 'cabeza' durante una inserción al inicio.

Porque si actualizamos la cabeza primero, rompemos el enlace con el resto de la estructura existente,

provocando que se pierdan los nodos subsiguientes.

---

10. ¿Qué sucede con la memoria en Java si desconectas accidentalmente el primer nodo de una lista sin haber guardado una referencia al resto de los elementos? Justifica tu respuesta mencionando el concepto de *Garbage Collector*.

Si desconectas el primer nodo sin conservar una referencia al resto de la secuencia, toda la cadena

subsiguiente queda inaccesible (sin referencias apuntando a ellos). El *Garbage Collector* detectará estos

nodos como inalcanzables o "huérfanos" y procederá a liberar automáticamente su espacio de memoria en el *heap*.
